1A

SHANI "DILL" DESILVA
U14033199

| AB CD Value | INPUT A | B | C | D | Output F |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 2 | 0 | 0 | 1 | 0 | 1 |
| 3 | 0 | 0 | 1 | 1 | 0 |
| 4 | 0 | 1 | 0 | 0 | 0 |
| 5 | 0 | 1 | 0 | 1 | 1 |
| 6 | 0 | 1 | 1 | 0 | 0 |
| 7 | 0 | 1 | 1 | 1 | 1 |
| 8 | 1 | 0 | 0 | 0 | 1 |
| 9 | 1 | 0 | 0 | 1 | 0 |
| 10 | 1 | 0 | 1 | 0 | 1 |
| 11 | 1 | 0 | 1 | 1 | 0 |
| 12 | 1 | 1 | 0 | 0 | 0 |
| 13 | 1 | 1 | 0 | 1 | 1 |
| 14 | 1 | 1 | 1 | 0 | 0 |
| 15 | 1 | 1 | 1 | 1 | 1 |

(1b) $\sum m$ of $F' = [1, 3, 4, 6, 9, 11, 12, 14]$

$(\prod M$ of $F = \sum m$ of $F')$

If "this function" means for F,

$\prod M$ of $F = [1, 3, 4, 6, 9, 11, 12, 14]$

If "this function" means $F'$ in the previous sentence,

$\prod M$ of $F' = [0, 2, 5, 7, 8, 10, 13, 15]$.

added after asking Prof

The inverse of F is
$a'b'c'd + a'b'cd + a'bc'd' + a'bcd' + abc'd' + abcd' +$
$\hookrightarrow ab'c'd + ab'cd.$
which simplifies to $bd' + b'd$ using Kmap from 1c.

Shani 'DILL' De Silva
U1403199

Shani "Bill" De Silva
U1403199

(1c)

KMAP

| AB \ CD | 00 | 01 | 11 | 10 |
|---|---|---|---|---|
| 00 | 1 (0) | 0 (1) | 0 (3) | 1 (2) |
| 01 | 0 (4) | 1 (5) | 1 (7) | 0 (6) |
| 11 | 0 (12) | 1 (13) | 1 (15) | 0 (14) |
| 10 | 1 (8) | 0 (9) | 0 (11) | 1 (10) |

$B'D' + BD$

$a'b'c'd' + ab'c'd' + b'cd' + b'c'd + bcd + a'bd$

(0) (8) (2,10.) (5,13.) (7,15) (7,5.)

PIs: 0, 2, 5, 7, 8, 10

PIs ~~PIs~~ $a'b'c'd'$

QUINE MCCLUCKEY Proof

| EPIs. | 0 $a'b'c'd'$ | 2 $a'b'cd'$ | 5 $a'bc'd$ | 7 $a'bcd$ | 8 $ab'c'd'$ | 10 $ab'cd'$ | 13 $abc'd$ | 15 $abcd$ |
|---|---|---|---|---|---|---|---|---|
| BD | | | X | X | | | X | X |
| $B'D'$ | X | X | | | X | X | | |

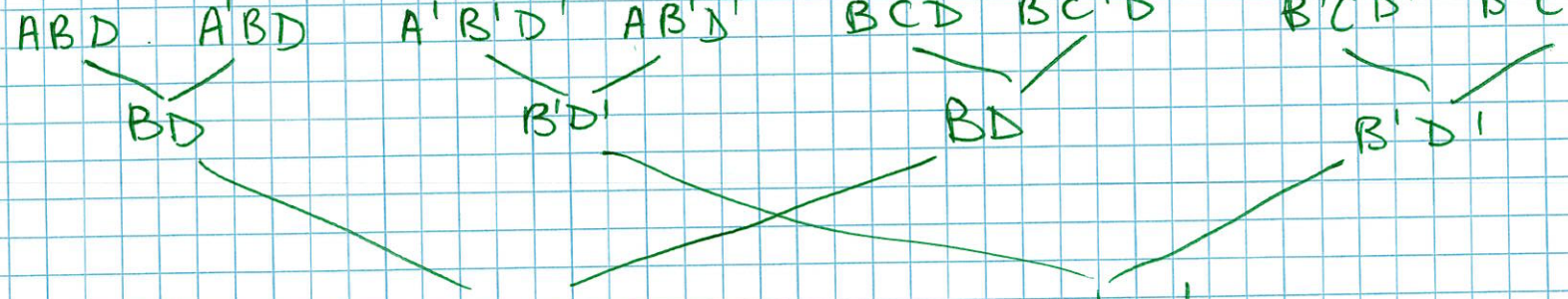
↑ from KMAP

Both BD & $B'D'$ are EPIs.

MINIMAL EXPRESSION = $BD + B'D'$.

1c, cont. QUINE MC CLUCKEY ABCD

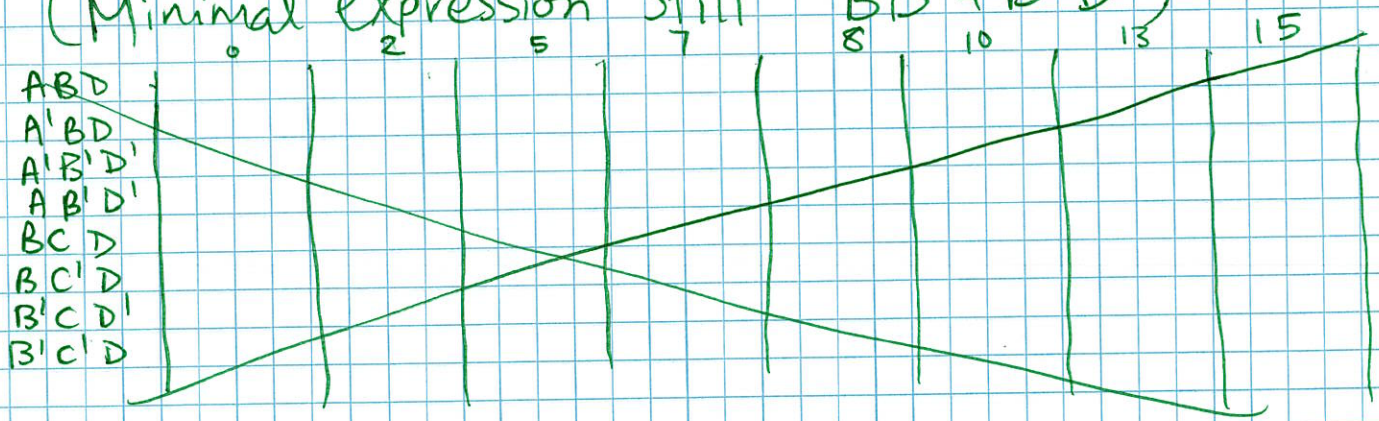
implicant options: ABD, A'BD, A'B'D', AB'D', ~~BCD~~, BC'D, B'CD', B'C'D'.




Quine McCluckey PIs: BD + B'D'.

| PI | ab'c'd' (0) | a'b'cd' (2) | a'bc'd (5) | a'b'cd (7) | ab'cd' (8) | ab'cd' (10) | abc'd (13) | abcd (15) |
|---|---|---|---|---|---|---|---|---|
| BD | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 |
| B'D' | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 |

(Minimal expression still BD + B'D')




Shani Dill De Silva
U1403319 9

Shani "Dill" De Silva
U14033199

(1d) EPIs: $BD$ & $B'D'$.

| | 0 | 2 | 5 | 7 | 8 | 10 | 13 | 15 |
|---|---|---|---|---|---|---|---|---|
| $BD$ | | | X | X | | | X | X |
| $B'D'$ | X | X | | | X | X | | |

Both $BD$ and $B'D'$ are EPIs because they contain at least 1 unique X in a minterm column so they are required to cover that minterm.

$BD$ is required to cover $[5, 7, 13, 15]$ and
$B'D'$ is required to cover $[0, 2, 8, 10]$.

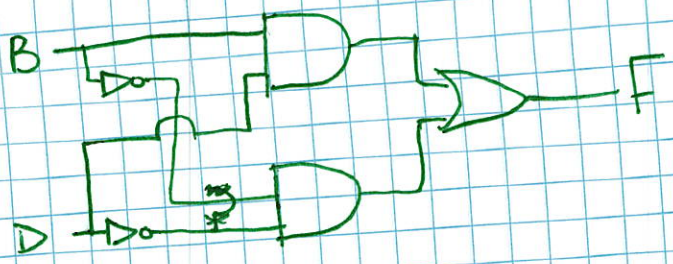
(1e) there are no static hazards in $BD + B'D'$ because there aren't any adjacent midterms not covered by the same EPI. no PIs need to be added.

Shani "Dill" De Silva
U14033199.

# Q1F

Shani 'Dill' DeSilva
U1403199.

logic circuit.

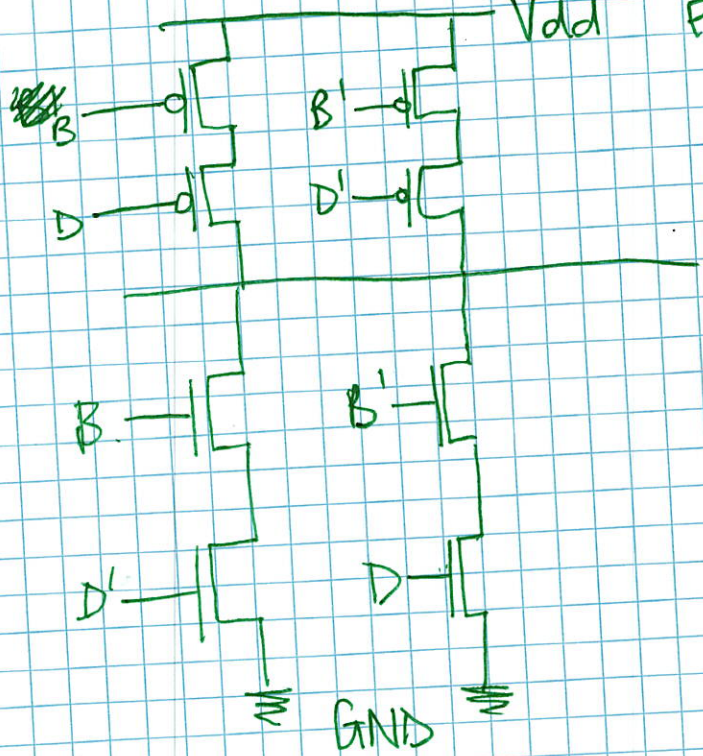



A, C } Not needed.

CMOS CIRCUIT:

$F = BD + B'D$

$F' = BD' + B'D.$



Shani "Dill" De Silva
U1403199

1g
1h
1i
1j

Answers typed & inserted into scan of answers.

Shani “Dill” De Silva, U14033199
Question 1G

Note: for verilog code, I am assuming built-in primitive gate outputs are always the 1st argument. (Source: https://chipverify.com/verilog/verilog-gate-level-modeling for documentation).

I kept a and c as inputs, despite them not being needed, as I thought the syntax of the inputs might already be defined.

This is a structural description, because it specifies the exact built-in operations, gates, and wires to be used and connected, allowing no room for xilinx-side interpretation.
==========

```
module exam_1g (out, a, b, c, d);
        input  a, b, c, d;
        output out;
        wire not_b, not_d, bd, notbnotd;

        not notb (not_b, b);
        not notd (not_d, d);
        and b_and_d (bd, b, d);
        and nb_and_nd (notbnotd, not_b, not_d);
        or bd_or_nbnd (out, bd, notbnotd);

endmodule
```

Shani “Dill” De Silva, U14033199
Question 1H

Note: for verilog code, using https://chipverify.com/verilog/verilog-assignments#continuous and https://coddy.tech/learn/verilog/fundamentals/continuous_assignment as reference for syntax.

I kept a and c as inputs, despite them not being needed, as I thought the syntax of the inputs might already be defined.

This is a behavioural description, as whenever b or d change, the output is updated immediately using maths terms instead of specified structural gates. The actual gate implementation is left to the simulator/synthesiser.
==========

```
module exam_1h (out, a, b, c, d);

        input wire a, b, c, d;
        output wire out;

        assign out = (b&d) | ((~b)&(~d));

end module
```

Shani “Dill” De Silva, U14033199
Question 1I

Note: for verilog code, using https://chipverify.com/verilog/verilog-always-block as reference for syntax.

I kept a and c as inputs, despite them not being needed, as I thought the syntax of the inputs might already be defined.

Like 1H, this is a behavioural description. Whenever b or d change, the output is updated immediately using maths terms instead of specified structural gates. The actual gate implementation is left to the simulator/synthesiser. However, this module uses an always block instead of an ‘assign’.
==========

```
module exam_1h (out, a, b, c, d);

        input wire a, b, c, d;
        output reg out;

        always @(b or d) begin
                out = (b&d) | ((~b)&(~d));
        end

end module
```

Shani “Dill” De Silva, U14033199
Question 1J (references 1H)

Note: for verilog code, using https://chipverify.com/verilog/verilog-combinational-logic-assign as reference for syntax.

I kept a and c as inputs, despite them not being needed, as I thought the syntax of the inputs might already be defined.
==========
1H code:

```
module exam_1h (out, a, b, c, d);

        input wire a, b, c, d;
        output wire out;

        assign out = (b&d) | ((~b)&(~d));

end module
```
==========

To make 1H unsynthesizable, without using delays or syntax errors, I could split up the OR statement to double-drive the out-wire; I suspect that the synthesizer would have an issue with the two outputs on one wire even if technically it could be done?

```
module exam_1j (out, a, b, c, d);

        input wire a, b, c, d;
        output wire out;

        assign out = (b&d);
        assign out =  (~b)&(~d);

end module
```

(2a)

Shani "Dill" DeSilva
U14033199

| Taken | CS1, CS0 | NS1, NS0 | Predict. |
|---|---|---|---|
| 0 | 0 0 | 0 1 | 1 |
| 0 | 0 1 | 1 1 | 1 |
| 0 | 1 0 | 1 1 | 0 |
| 0 | 1 1 | 1 1 | 0 |
| 1 | 0 0 | 0 0 | 1 |
| 1 | 0 1 | 0 0 | 1 |
| 1 | 1 0 | 0 0 | 0 |
| 1 | 1 1 | 1 0 | 0 |

# 2B: FSM controller diagram

Shani "Dill" DeSilva
U14033199

diagram

| Taken | Cs1 | Cs0 | Predict |
|---|---|---|---|
| 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 |

← not affected by Taken or Cs0.

Predict = (Cs1)'

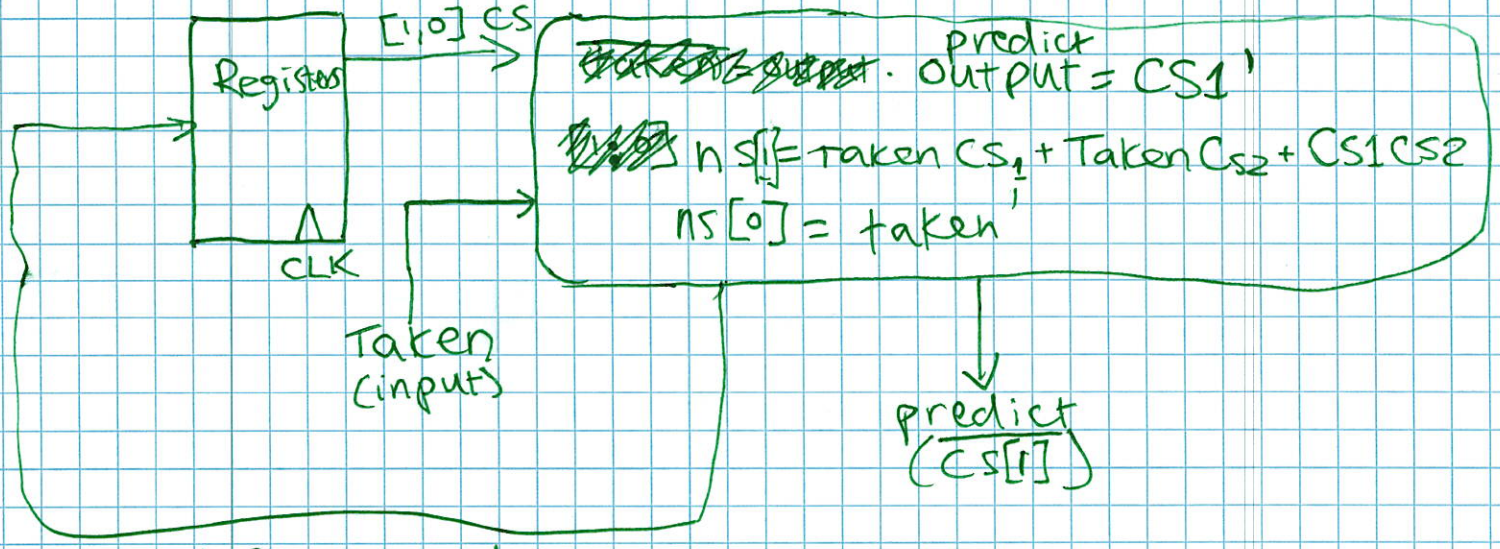
| Taken | Cs1 | Cs0 | Predict | Ns1 | Ns2 |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 0 | 0 | 1 | 1 |
| 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 0 | 1 | 0 |

Ns1 ↑ Taken ≠ (CS1 OR CS2)

Ns2 ↑ taken'

*

Taken·Cs1 + Taken·Cs2 + Taken' Cs1 Cs2 · { Taken ≠ (CS1 OR CS2) / (Taken' or and (cs1 and Cs2).




NS[0] = taken'

NS[1] = Taken · CS1 + Taken CS2 + CS1 CS2.

* TC1 + TC2 + T̄C1C2.
= TC1 + TC1C2 + TC2 + T̄C1C2
= TC1 + TC2 + C1C2(T + T̄)
= TC1 + TC2 + C1C2.

used info from:
cs.cornell.edu/courses/cs3410/2018fa/schedule/slides/07-fsm.pdf .

Shani "Dil" De Silva
U1403319 9

(2c) Answers typed & inserted into scan of PDF.

Shani “Dill” De Silva, U14033199
Question 2C
Create a behavioral Verilog description for the FSM using the template provided.
Assuming clock-synchronous reset on high reset input.

syntax references used:
https://chipverify.com/verilog/verilog-always-block
https://coddy.tech/learn/verilog/fundamentals/concatenation_operator for sanity check
https://chipverify.com/verilog/verilog-case-statement
==========

```
module state_machine (clk, reset, taken, predict);

        input clk, reset, taken;
        output predict;

        reg [1:0] cs; //use bit width and number of variables as desired
        //using the register cs to store states 0 to 3.


        //Fill in….
        always @(posedge clk) begin
                if(reset) begin
                        cs = 2'b00; //reset state to 00.
                end else begin
                        case ({taken, cs}) begin //3 to 2 mux
                                3'b000: begin
                                        cs = 2'b01;
                                        predict = 1'b1;
                                end
                                3'b001: begin
                                        cs = 2'b11;
                                        predict = 1'b1;
                                end
                                3'b010, 3'b011: begin
                                        cs = 2'b11;
                                        predict = 1'b0;
                                end
                                3'b100, 3'b101: begin
                                        cs = 2'b00;
                                        predict = 1'b1;
                                end
                                3'b110: begin
                                        cs = 2'b00;
```

```
                predict = 1'b0;
            end
            3'b111: begin
                cs = 2'b10;
                predict = 1'b0;
            end
        endcase
    else
end
endmodule
```

Shani "Dell" De Silva
U1403S199

2D

Option 1:
I could have used nested IF statements instead of the case statement, but doing so would have increased the complexity of the synthesis. A case statement is essentially a Mux - an if statement is not as simple.

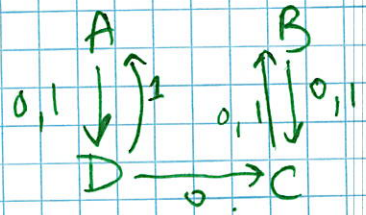
Option 2:
I could have written the case statement to take only 'taken' or only 'cs' as an input, and created nested case statements. this is redundant - ~~wha~~ despite the code's logic being more readable, it is less efficient (if it isn't synthesised out!).

Option 3:
I could have ~~taken~~ the inputs & current state values ~~to create~~ and computed the boolean logic gate circuits needed to arrive at the output state & prediction. This would have taken me far more time to write, which is a tradeoff in itself.
(did 2c ~~before~~ 2B)

(2e)

Group 1: B, C.

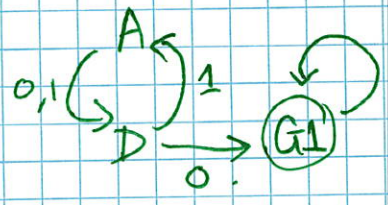
Group 2: A, D.




For group 1,

B 0 → C 1
B 1 → C 0
C 0 → B 1
C 1 → B 1

} G1 to G1. same.




For group 2,

A 0 → D 1
A 1 → D 1
D 1 → A 1
D 0 → G1 ← different.

Change G2 to be just A and G3 to be just D.

for group 1, G1 → G1.

for group 2, G2 → G3 + one state group.

for group 3, G3 → G2
→ G1

but it's fine as a 1 state group.

| State | input 0 | input 1 |
|---|---|---|
| A | D1 | D1 |
| X | XX | XX |
| D | A1 | XX. |

input (Don't care = X).

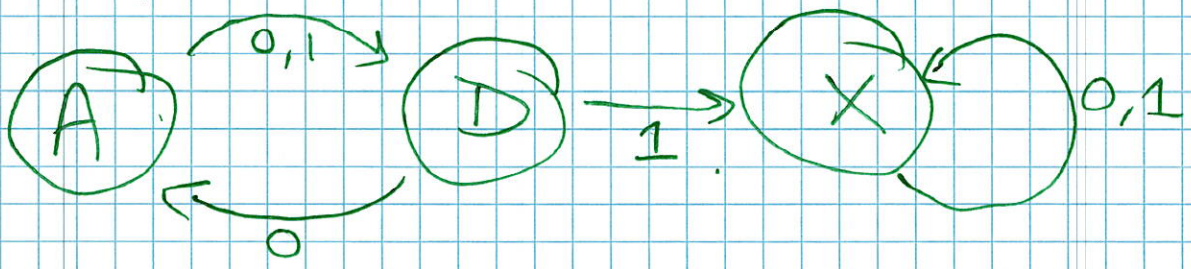

Shani "Dill" De Silva

U1403319 9

Shani "Dill" De Silva
U1403 3199.

(2f)

| State | input 0 | 1 |
|---|---|---|
| A | D,1 | D,1 |
| X | X,X | X,X |
| D | A1 | X,X. |

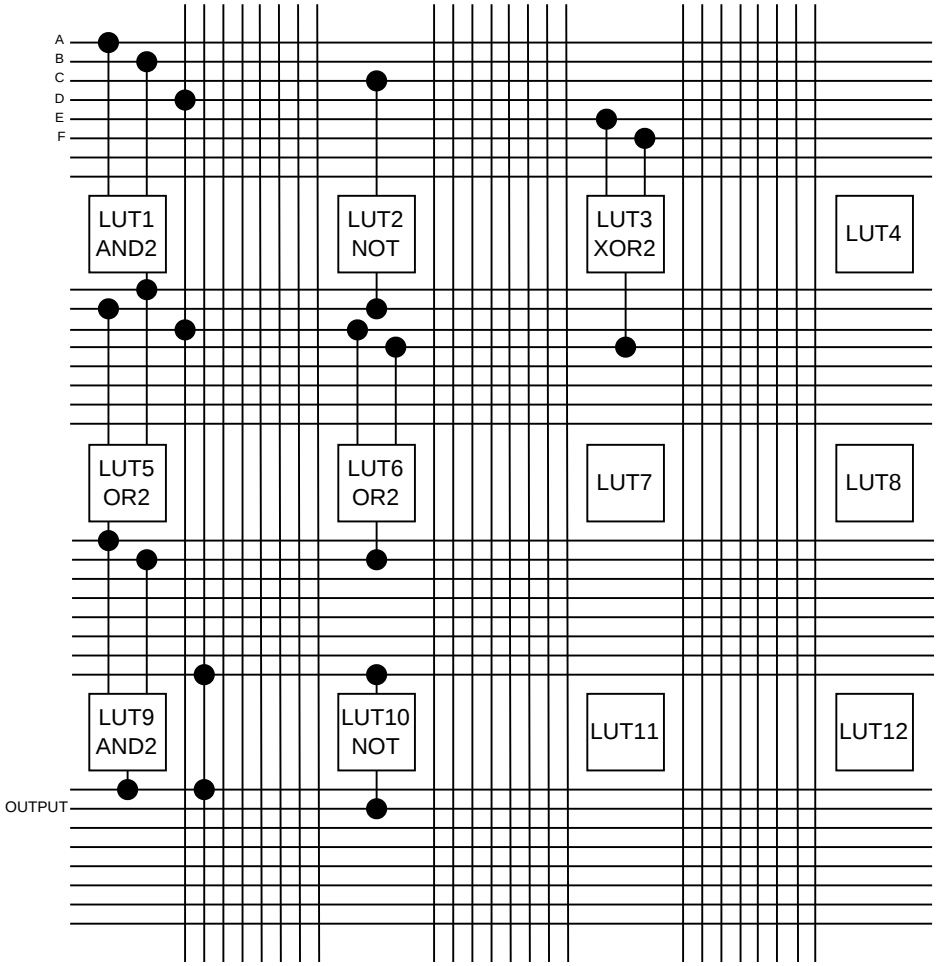



This is a Mealy machine - all outputs are functions of the current state combined with an input.

Shani "Dill" DeSilva
U1403 3199.

(3a) Question answered on draw.io, for cleaner diagram, and will be inserted into pdf.

Shani "Dill" De Silva
U14033199
Q3A




LUT1: AND 2
LUT2: NOT
LUT3: XOR2
LUT 4:
LUT 5: OR2
LUT 6: OR2
LUT7:
LUT8:
LUT9: AND2
LUT10: NOT
LUT11:
LUT12:

HORIZONTAL
LINE0: A, LUT1 IN0
LINE1: B, LUT1 IN1
LINE2: C, LUT2 IN1
LINE3: D, LINEA
LINE4: E, LUT3 IN0
LINE5: F, LUT3 IN1
LINE6:
LINE7:

HORIZONTAL
LINE8: LUT1 OUT, LUT5 IN0
LINE9: LUT2 OUT, LUT5 IN1
LINE10: LINEA, LUT6 IN0
LINE11: LUT3, LUT6 IN1
LINE12:
LINE13:
LINE14:
LINE15:

HORIZONTAL
LINE16: LUT5 OUT, LUT9 IN0
LINE17: LUT6 OUT, LUT9 IN1
LINE18:
LINE19:
LINE20:
LINE21:
LINE22:
LINE23: LINEB, LUT10 IN

HORIZONTAL
LINE24: LUT9 OUT, LINEB
LINE25: LUT10 OUT, OUTPUT
LINE26:
LINE27:
LINE28:
LINE29:
LINE30:
LINE31:

VERTICAL
LINEA: LINE3, LINE10
LINEB: LINE23, LINE24
LINEC:
LINED:
LINEE:
LINEF:
LINEG:
LINEH:

VERTICAL
LINEI:
LINEJ:
LINEK:
LINEL:
LINEM:
LINEN:
LINEO:
LINEP:

VERTICAL
LINEQ:
LINER:
LINES:
LINET:
LINEU:
LINEV:
LINEW:
LINEX:

Shani "Dill" De Silva
U14033199

(3B) If all LUTs had to be 4-input functions & all line-line connections could be made:

My gate:
- 7 LUTs used.
- 15 horizontal lines used
- 2 vertical lines used (inter-LUT columns)

However! Assuming Question means ~~[illegible]~~ value of SRAM needed to store function/connection when gates fully used:

(if) 16 bits per 4 input LUT.

(if) 12 total LUTs

(if) vertical/horizontal wire intersections are governed by switch box matrix.

For 12 LUTs with 4 inputs = 192 bits. storage.
↳ 16 bits to store answers

Routing wires: 32 horizontal, 24 vertical, $32 \times 24 = 768$ bits

5 pins on each LUT, 12 LUTs: 60 connections to wires.

however! 8 options each $(8 \times 4 + 8)12 = 480$ bits.
($40 \times 12$)




$\geq$

12 LUTs x 5 Pins x 8 options per pin.

$12 \times 40 = 480$.

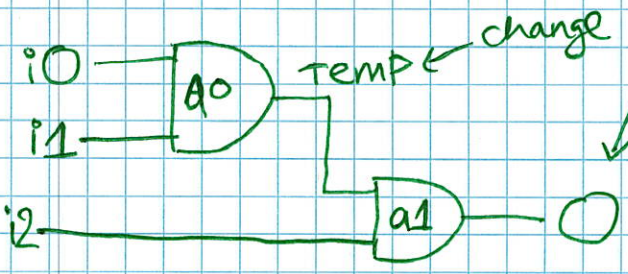
$192 + 768 + 480 = 1440$ bits

Shani 'Dill' De Silva

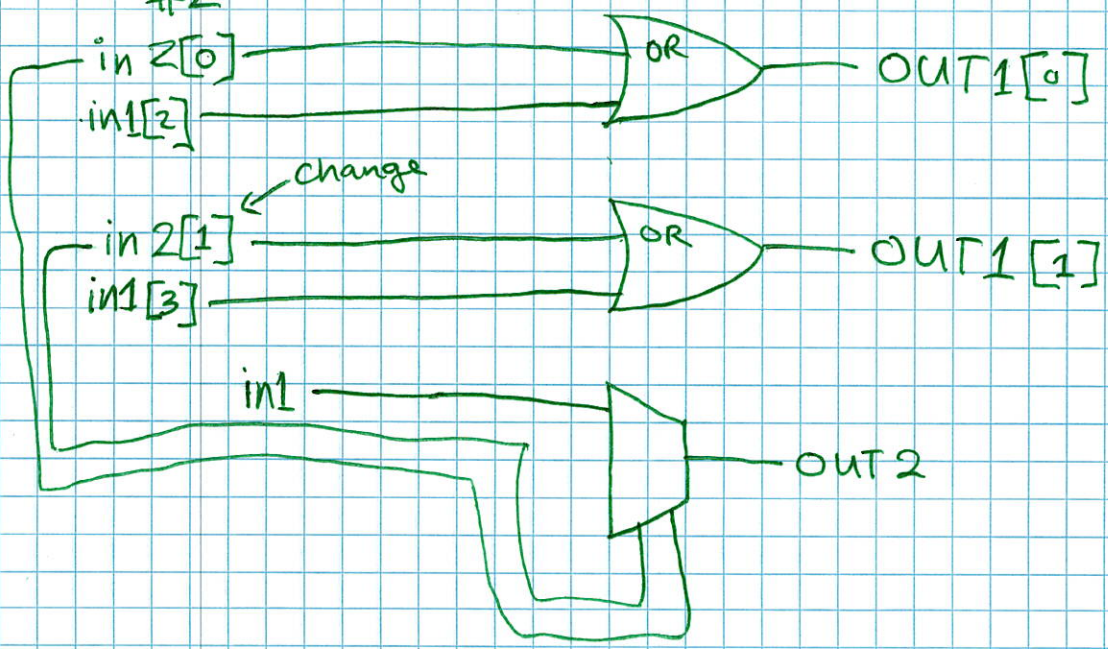
U14053199

Q4 #1

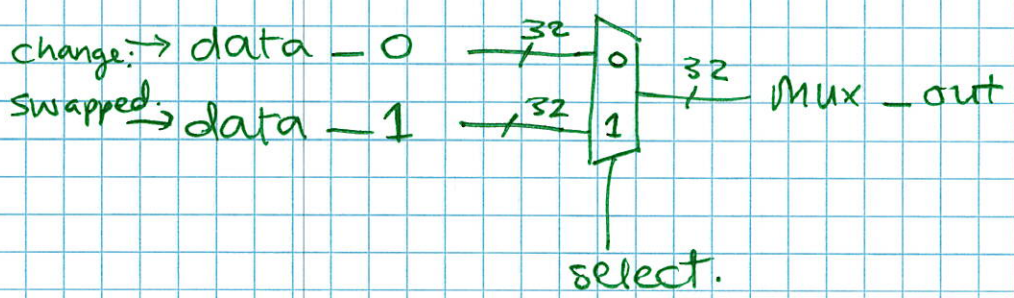
AND syntax = input input output I am assuming? not primitive gate or order would be wrong and uppercase letters would be wrong. only one error.




#2




#3 condition? true:false.




Typos:

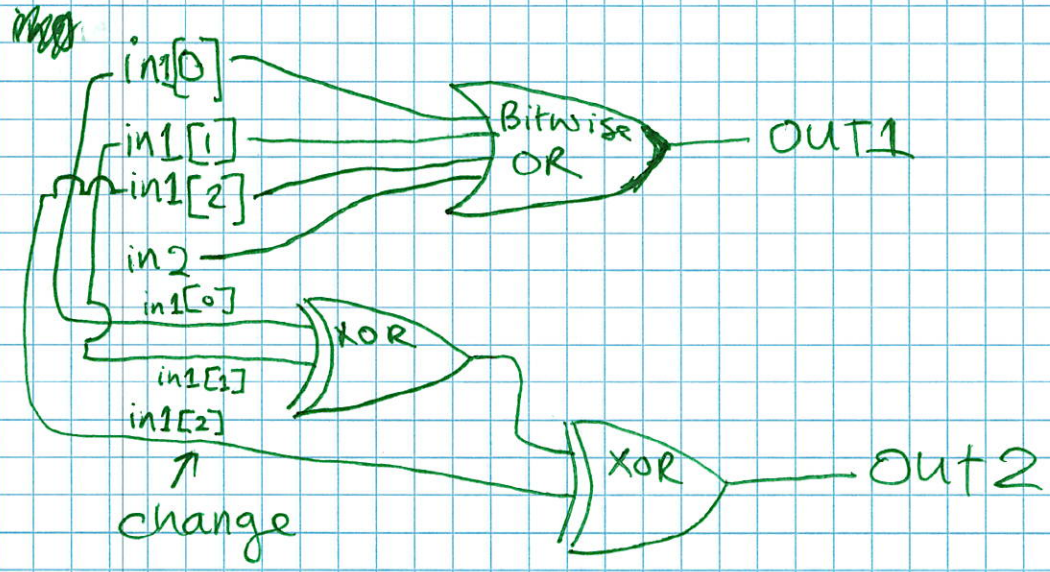
data0 doesn't exist - should be data_0.

mux-out (hyphen) won't work - assuming it outputs mux_out.

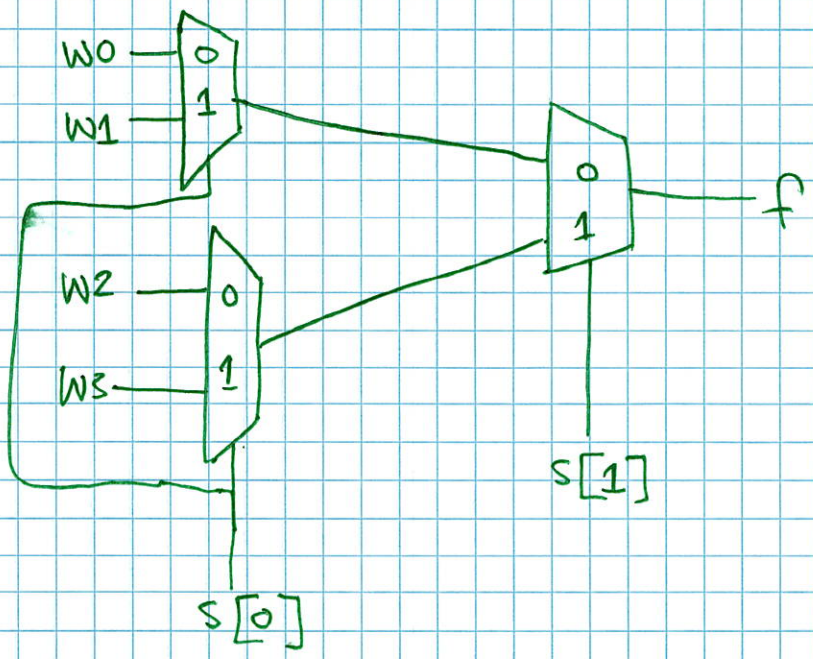
Q4, cont

#4

Shani "Dill" DeSilva
U1140331 99




#5




(To be quite honest I'm not sure which part of the code → Diagram was wrong)

Q4, cont) The 5 paper questions & answers have been typed & will be inserted.

Q5 all of Q5 is typed and will be inserted as my hands are not doing good (accommodation stuff I am very sorry)

Shani "Bill" DeSilva

U140331 99

Choosing to type this answer as my hands are starting to not work (accommodations include this I think?); my apologies for the change in format!

1. What is the “technology independent” phase of logic synthesis?

The “technology independent phase of logic synthesis describes the point in design development that is not dependent on the technology chosen, IE where the code will be written or what chip the code will be run on. It is quite early on in the logic synthesis process, as it is done pre-synthesis and pre-anything concrete!

2. What is the cost metric used for a circuit?

The cost metric used to value a circuit is the “sum of the literals in the factored form of the node expressions”.

“The cost given is that of the resulting network, as measured by the literal count of the nodes expressed in factored form.” - The Synthesis of Cyclic Combinational Circuits paper
“For each node, we expect the lowest cost expression to be obtained with the full substitutional set (i.e., all other node functions) and the highest cost expression to be obtained with the empty set” -The Synthesis of Cyclic Combinational Circuits paper

I can’t find anywhere else to mention how entertained I am by the concept of the Jiggle Algorithm mentioned in a cited paper.
R. K. Brayton, G. D. Hachtel, C. T. McMullen, and A.
L. Sangiovanni-Vincentelli, “Multilevel Logic
Synthesis,” Proceedings of the IEEE, Vol. 78, No. 2,
p. 264 – 300, 1990.

3. Describe substitution.

“Node functions are expressed, or re-expressed, in terms of other node functions as well as of their original inputs.”-The Synthesis of Cyclic Combinational Circuits paper

4. Why would you want to get rid of cycles in combinational circuits?

“The behavior of a circuit with feedback is generally more complicated. Such a circuit may exhibit timing-dependent behavior (as in the case of an R-S Latch), and it may be unstable (as in the case of an oscillator).”-The Synthesis of Cyclic Combinational Circuits paper

If there is no specific benefit for the circuit to be cyclic, then it is better for the circuit to be simpler, “loop-free”, and more “clearly combinational”. -The Synthesis of Cyclic Combinational Circuits paper

5. What is topological vs. electric feedback?

# Q4 tech paper question 5

Topological feedback refers to a visible loop in logic that is nevertheless ironed out when real logic values are introduced; see the ADECA cycle in Figure two, but how it is smoothed out by adding values in Figure 4.

Electrical feedback in a cyclic circuit refers to how outputs may not settle on an electrical value due to feedback from a loop, called the “Up-Bounded inertial delay model” citing J. A. Brzozowski and C.-J. H. Seger, “Asynchronous Circuits,” Springer-Verlag, 1995.

(Figures from The Synthesis of Cyclic Combinational Circuits paper)

Shani “Dill” De Silva, U14033199

Q5

a. Verilog testbenches should be written so that they can be synthesized along with the DUT.
    i. False - Verilog testbenches are not synthesizable - they are only used to provide inputs for a module and can’t be loaded onto an FPGA.
b. Always blocks are only used for sequential logic design.
    i. False - I think that if all values are listed in the sensitivity list, and there are no points where the output is dependent on memory, an always block can technically be combinational? https://chipverify.com/verilog/verilog-always-block
c. When writing Verilog code, it is important to know about the hardware you are targeting to get the best synthesis result.
    i. True
d. FPGAs that are larger will always require more power but will allow you to have more complex designs.
    i. False - I think the power used also depends on how much of the FPGA is actually used? Larger FPGAs will let you run more complicated designs though, that bit is correct.
e. Non-blocking statements are better for modeling hardware. If you are going to synthesize a design you shouldn’t use blocking statements.
    i. False; blocking and non-blocking assignments are used for different things. The important thing here is to not use one when you mean the other!
f. Fix the following 5 Verilog code segments. These changes could be syntactic or functional. In the event you see multiple errors, choose the most functionally relevant (or mention all).

F1:

```
//Do not want an accidental synthesis of a latch
always@(posedge clk)
        casez (op)
                2’b1? : // assignment block 1
                2’b?1: // assignment block 2
        endcase
```

Error: no ‘end’ shown in always block.
Warning: 00 has no case and so data will remain unchanged, and 11 will only run the first case; not wrong, but it is annoying!

---

F2:

```
wire A, B, C
always @ (posedge clk)
begin
        A=C;
```

```
        B=A;
        C=B;
end
```

Error 1: this code, when run, has three competing lines of code that can get completed in different orders each time, changing the output of the block. Would introduce temporary memory or use blocking instructions.

---

F3:

```
Module foo (s, A, B, C, Z);
        input A, B, C;
        output Z;
        input [1:0] s;
        always @ (s or A or B or C)
                case (s)
                        3’b100 : Z<= A;
                        3’b010 : Z<= B;
                        3’b001 : Z<= C;
                endcase
endmodule
```

Error 1: the input s is 2 digits wide; the values that the case statement is looking for are 3 digits long. Maybe the person intended to use the values 3, 2, 1 instead of the values 4, 2, 1?

---

F4:

```
Module adder_4_bit (A, B, C_in, Sum, C_out);
        Input [3:0] A, B;
        Input C_in;
        Output [3:0] Sum;
        Output C_out;
        Wire [3:0] A, B, C_in, Sum, C_out;

        fulladder FA3(A[3], B[3], C2, Sum[3], C_out);
        fulladder FA2(A[1], B[2], C_in, Sum[2], C2);
        fulladder FA1(A[2], B[1], C0, Sum[1], C1);
        fulladder FA0(A[0], B[0], C1, Sum[0], C0);
endmodule
```

Error 1: I believe that for fulladder FA2 and FA0, the C_in inputs are switched; they should read:

```
        fulladder FA0(A[0], B[0], C_in, Sum[0], C0);
        fulladder FA2(A[1], B[2], C1, Sum[2], C2);
```

---

# Q5

F5:

```
//Should be a truly combinational circuit
always @ (x or y)
begin
w = y & z;
u = w | x;
end
```

Error 1: ‘z’ is missing from the always sensitivity list, assuming that this is the only place where w and u are changed. w is updated by y and z, u is updated by w and x, but z is not on the sensitivity list.

g. There is a group that is planning on creating a driving simulation game for their project.
    i. False - I don’t believe anyone is planning on making a game at all. The projects as far as I’m aware are:
    FPGA Qubit Controller
    Ray Tracing Core
    Secure Computing Enclosure using Sequestered Encryption
    FPGA-Based Real-Time Vital-Signs Monitor
    Live Audio Frequency Visualizer
    FPGA-Based Finite Impulse Response (FIR) Filter
    Image processing pipeline
    HFT-FPGA System
    (referenced from https://piazza.com/class/mtd4fkguc682l9/post/16)
h. Using the “reg” syntax must lead to a register being synthesized.
    i. False: sometimes the register that is simulated will get synthesised out.
i. Circle the correct ordering of the digital design flow. (iii)
    i. Design spec -> simulation -> design entry -> design partition -> synthesize
    ii. Design spec -> design entry -> design partition -> simulation -> synthesize
    iii. Design spec -> design partition -> design entry -> simulation -> synthesize
    iv. Design partition -> design spec -> design entry -> simulation -> synthesize
j. Verilog uses “Event Driven Simulation”.
    i. True
k. How many functions can a 4-input LUT represent? (iv)
    i. 16
    ii. 4
    iii. Infinite
    iv. 65536 (16 possible output slots, each can either be 1 or 0, so 2^16)
l. We read two papers in class related to cyclic combinational logic circuits.
    i. What were each about? (2-3 sentence for each paper)
        1. The Cyclic to Acyclic paper described how to convert the combinatorial portions of a circuit (for inputs where the output doesn’t oscillate or require stored memory) into a separate acyclic circuit. This was done by

creating and then merging ‘acyclic fragments’. The author presents a proof of the statement “For a circuit with a strongly-connected component (SCC) to behave combinationally, at least one input to a gate in the SCC must be driven to a controlling value.”, and uses said proof to demonstrate examples of cyclic-to-acyclic conversion. They discuss their prototype to automate this, and mention future work.

2. The Acyclic to Cyclic paper describes the benefits of synthesising an apparently cyclic circuit, despite its generally more complicated behaviour, and makes clear the difference between topological feedback and electrical feedback in circuits and the importance of non-oscillation. The author goes through their algorithm to create small loops to minimise work done, presenting the “Build Up Approach” and the “Break-Down Approach”. They look at the cost of equivalent cyclic and acyclic circuits, where the cost metric is measured by the number of literals used. They end by calling for a paradigm shift in the field.

ii. Provide two questions you made for each paper for the paper game.

1. Cyclic to Acyclic: https://github.com/ddsva1/bu-EC551/blob/main/required-reading/making-cyclic-circuits-acyclic-questions.txt
    a. How does the merging algorithm avoid creating a cycle?
    b. How do we choose which acyclic circuit fragments to discard?
2. Acyclic to cyclic: https://github.com/ddsva1/bu-EC551/blob/main/required-reading/synthesis-of-cyclic-combinational-circuits.pdf
    a. What was Malik's work on?
    b. What is a substitutional set?

m. What are suggestions you have for improving the class?

i. Of course a bigger, closer-to-Engineering, better room than CGS 121 would be preferred! I can’t help but think that it would be easier for you to teach and for us to see the board in a different room.

ii. I would prefer smaller individual projects instead of doing the ‘individual’ courseworks in teams of 2 with the amount of work scaled up.

It was hard to find and organise a team for this, if you don’t come into the class with teams you are accustomed to from previous courses, which is leaving my team with not enough time to complete the coursework!

Ideally, I’d prefer the ability to enter a random pairing of names of people who don’t have a preferred team at the start of the project or smaller, genuinely individual projects.

iii. If you have covid recordings for the lectures, would it be possible to upload those to blackboard to provide another way to revise? If not, absolutely fine

iv. Great format for the exam!

# Q5